Oracle-Guided SMT Solver Validation via Synthesizing Diverse Formulas with Multiple Assignments

MAOLIN SUN, YIBIAO YANG, JIANGCHANG WU, QINGYANG LI, ZEYU LU, and YUMING ZHOU, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

Ensuring the correctness of Satisfiability Modulo Theory (SMT) solvers is critical, as undetected bugs in these solvers can lead to incorrect results in the many applications that depend on them. However, automatically testing SMT solvers remains challenging, largely due to the absence of reliable test oracles. To address this challenge, recent work has explored assignment-aided reassembling techniques, such as STORM and Diver, which generate satisfiable test formulas by recombining parts of existing formulas based on satisfying assignments. While these techniques have demonstrated effectiveness in practice, opportunities remain to further enhance the approaches. For example, they cannot generate unsatisfiable test formulas, which may restrict their ability to detect certain classes of soundness bugs. Therefore, this article introduces a novel fuzzing technique that enhances these methods building on the idea of assignment-aided reassembling. Specifically, our approach utilizes multiple satisfying assignments derived from seed formulas to guide the reassembly of sub-formulas, exploring divergent logical paths to synthesize new test cases. To further increase test diversity, we incorporate formula snippets generated from scratch. In addition, we extend the approach to support the generation of unsatisfiable test formulas by using unsatisfiable cores extracted from unsatisfiable seed formulas. We have implemented this approach in a practical tool, Octopus, designed to test SMT solvers more comprehensively. We applied Octopus to leading SMT solvers, including Z3 and cvc5, and reported 13 bugs to their issue trackers. Among these, 11 bugs have been confirmed by the developers, and nine have been fixed. Our experiments also show that Octopus achieves higher code coverage compared to existing SMT fuzzers.

CCS Concepts: • **Software and its engineering → Software testing and debugging**;

Additional Key Words and Phrases: SMT solver, fuzzing, sub-formulas, assignment-aided reassembling

This work was supported in part by the National Natural Science Foundation of China (Grants 624B2067, 62572226, 62472215, and 62072194), the Jiangsu Natural Science Foundation (Grant BK20231402), and the Collaborative Innovation Center of Novel Software Technology and Industrialization.

Authors' Contact Information: Maolin Sun, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: merlin@smail.nju.edu.cn; Yibiao Yang (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: yangyibiao@nju.edu.cn; Jiangchang Wu, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: jiangchangwu@smail.nju.edu.cn; Qingyang Li, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: liqingyang@smail.nju.edu.cn; Zeyu Lu, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: zeyulu@smail.nju.edu.cn; Yuming Zhou, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: zhouyuming@nju.edu.cn.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 1557-7392/2026/9-ART292
https://doi.org/10.1145/3788862

ACM Reference format:
Maolin Sun, Yibiao Yang, Jiangchang Wu, Qingyang Li, Zeyu Lu, and Yuming Zhou. 2026. Oracle-Guided SMT Solver Validation via Synthesizing Diverse Formulas with Multiple Assignments. *ACM Trans. Softw. Eng. Methodol.* 35, 9, Article 292 (September 2026), 29 pages.
https://doi.org/10.1145/3788862

1 Introduction

Satisfiability Modulo Theory (SMT) solvers are indispensable tools for verifying the satisfiability of logical formulas across various theories, including arithmetic, arrays, uninterpreted functions, strings, and their combinations [8]. These solvers are foundational in formal methods, underpinning prominent software verification tools such as Boogie [5] and Dafny [26]. For instance, Amazon employs SMT solvers to support service access control [53]. Additionally, SMT solvers are crucial for test case generation, as demonstrated by tools like KLEE [14] and Sage [19]. Given their critical role, any bugs or inaccuracies in SMT solvers can have severe consequences, potentially leading to misleading results in client applications or even security vulnerabilities [21]. For example, if a solver produces incorrect results, a symbolic execution engine might mistakenly conclude that a specific code path is unreachable, thereby failing to generate the necessary test inputs to explore that path. Thus, ensuring the reliability of SMT solvers is of paramount importance.

Motivated by these concerns, recent research has explored differential testing and oracle-guided fuzzing techniques to validate SMT solvers [10, 34, 40, 46–48, 53, 54]. Differential testing runs multiple SMT solvers on the same input formula and flags inconsistencies in their results as potential bugs. For example, ET [46] introduced a grammar-based approach to generate test formulas for differential testing. Alternative studies devise various mutation strategies to generate diverse test inputs and identify bugs through differential testing [32, 40, 47]. Although effective in many scenarios, differential testing faces challenges when test formulas involve solver-specific extensions, as differences in solver implementations may lead to discrepancies that are not true bugs. For example, the cvc5 solver [1] supports certain non-standard or extended theories, making direct comparisons with other solvers infeasible. Additionally, differential testing is inapplicable when solvers fail to return a result, e.g., non-termination. In contrast, oracle-guided testing techniques generate input formulas with known satisfiability to validate SMT solvers [13, 23, 28, 48, 54]. In this context, the test oracle provides an expected result indicating the satisfiability of a given formula. Notable approaches include assignment-aided reassembling techniques such as STORM [28] and Diver [23]. For instance, STORM begins by solving a seed formula using an SMT solver to obtain an assignment (*a.k.a.* solution or model), a mapping of values to variables, such as "$\mathcal{A} : a := true \wedge x := 5 \wedge y := 0$," representing an assignment for the formula shown in Listing 2. It then extracts sub-formulas (*a.k.a.* subterms) from the seed formula and evaluates their truth values under the obtained assignment. By combining these sub-formulas and their corresponding truth values, STORM synthesizes new satisfiable formulas to test the solver. A potential bug is reported if the solver incorrectly returns *unsat* for a satisfiable synthesized formula, indicating a *refutational soundness bug*. This type of bug arises when the solver determines a formula to be unsatisfiable while it is actually satisfiable. Conversely, a *solution soundness bug* occurs if the solver incorrectly returns *sat* for an unsatisfiable input. Soundness bugs are particularly critical, as they undermine the solver's reliability, and are a key focus for SMT solver developers.[1] In addition to soundness issues, other bugs, such as crashes, also affect solver robustness.

[1] https://github.com/cvc5/cvc5/wiki/Fuzzing-cvc5.

Table 1. Truth Values of the Sub-Formulas from the Original Formula φ (Listing 3) and the Modified Formula φ' (Listing 4) under Different Assignments

	$\mathcal{A} : a := T \wedge x := 5 \wedge y := 0$	$\mathcal{A}' : a := T \wedge x := 2 \wedge y := 2$
φ_1	T	T
φ_2	F	T
φ_3	F	T
φ_4	F	T
φ_5	T	F
φ_6	T	T
φ'	F	T

While assignment-aided reassembling techniques are effective for validating SMT solvers, they face inherent challenges in balancing input diversity with the availability of reliable test oracles. For example, in STORM, each extracted sub-formula is assigned a truth value, either `true` or `false`, based on a single assignment obtained from the solver. When a sub-formula evaluates to `false` under this assignment, STORM must negate it to ensure the constructed formula remains satisfiable. As a result, the original (unnegated) sub-formula cannot be used for generating satisfiable test inputs. This reliance on a single assignment limits the exploration of alternative formula combinations and reduces the flexibility of constructing test cases with known satisfiability. To illustrate, given a formula `x>0`, STORM requests an assignment from the solver and evaluates the truth value accordingly. If the solver provides the assignment `x=1`, the formula `x>0` evaluates to `true` and can be used for validation. However, its negation, `¬(x>0)`, evaluates to `false` under the same assignment and therefore cannot be used. Therefore, if the negated formula could reveal solver bugs, it would inevitably be overlooked due to the lack of an accurate oracle. A similar situation arises in Diver [23], which also relies on a single assignment to guide the synthesis of satisfiable formulas. Additionally, because both approaches focus solely on generating satisfiable formulas, they are unable to expose solution soundness bugs, which occur when a solver incorrectly returns *sat* for an unsatisfiable formula.

Approach. Building on the foundation of assignment-aided reassembling techniques, we propose Octopus,[2] a general approach designed to generate a diverse set of test formulas for validating SMT solvers. Octopus extends existing techniques in two key aspects. First, rather than relying on a single assignment from a seed formula, Octopus leverages *multiple assignments* of the same formula. This is motivated by the observation that sub-formulas can have different truth values under different assignments. For example, Table 1 shows how assignments $\mathcal{A}$ and $\mathcal{A}'$ for formula φ (from Listing 2) produce different truth values for the extracted sub-formulas. By utilizing multiple assignments, Octopus increases the likelihood of generating varied test formulas through reassembly. To improve efficiency, Octopus selectively uses assignments that yield differing truth values across sub-formulas, enabling the construction of more diverse test inputs. Second, Octopus introduces additional *formula snippets* to further expand the diversity of generated formulas, including both satisfiable and unsatisfiable instances. On one hand, we design a strategy for systematically generating new formula snippets, distinct from the sub-formulas extracted from the seed formula, based on the collected assignments. This expands the pool of building blocks

[2]Octopus = *O*ra*C*le-guided *T*esting thr*O*ugh synthesizing in*P*uts with m*U*ltiple a*S*signments.

```
1  (declare-fun v () Int)
2  (assert (and (= 2 v) (= v ((_ iand 1) v v))))
3  (check-sat)
```

Listing 1. Simplified formula triggering a solution soundness bug regarding the extended operator in cvc5.

available for synthesis. On the other hand, Octopus incorporates *unsatisfiable cores* [36], which are unsatisfiable sub-formulas responsible for making the seed formula unsatisfiable. By leveraging these unsatisfiable cores, Octopus can generate unsatisfiable test formulas to broaden testing coverage. In summary, Octopus integrates multiple assignments, systematic snippet generation, and the use of unsatisfiable cores to create a comprehensive framework for generating diverse test formulas for SMT solver validation.

We conducted extensive experiments to evaluate the effectiveness of Octopus. The results show that Octopus successfully identified 13 bugs in widely used SMT solvers, including Z3 [18], cvc5 [4], and Bitwuzla [31]. Of these, 11 bugs were confirmed by the developers, and nine have been fixed. Notably, most of these bugs are soundness bugs, which are of critical importance to the SMT solver community. In addition, Octopus demonstrates strong performance in efficiency, code coverage, and bug-finding capability compared to existing techniques.

Advantages over Existing Techniques. Compared to STORM, the multiple assignment component in Octopus improves the flexibility of formula construction, the use of unsatisfiable cores enables detection of solution soundness bugs, and the addition of novel formula snippets further increases the diversity of test cases. These components work together to make Octopus effective in uncovering bugs in SMT solvers. Additionally, unlike differential testing, which often fails when solver-specific features are involved or when reference solvers return `unknown` instead of a definitive answer, Octopus can detect such bugs by relying on a reliable test oracle. Listing 1 illustrates this advantage with a simplified bug-revealing formula involving the extended operator `iand` in `cvc5`. For this formula, the correct result should be `unsat`, but `cvc5` incorrectly reports `sat`. Because other solvers do not support the `iand` operator, differential testing cannot be applied, and the existing techniques, STORM and Diver, cannot expose the bug either since they are restricted to satisfiable formulas. This example highlights Octopus's unique ability to uncover bugs that existing methods struggle to detect.

Overall, the main contributions of this article are as follows:

- *Originality*. We propose a general framework, Octopus, to enhance assignment-aided reassembling approaches. Specifically, we leverage multiple assignments of a seed formula to generate more diverse test formulas with known satisfiability. We also introduce a systematic strategy to enrich the set of building blocks for formula synthesis with new formula snippets distinct from the original sub-formulas. Furthermore, we incorporate the use of unsatisfiable cores to generate unsatisfiable test formulas.
- *Usefulness*. We implement our approach as a practical tool, Octopus, for fuzzing SMT solvers. We compare Octopus with the **State-of-the-Art (SOTA)** oracle-guided fuzzer STORM and show that Octopus achieves higher code coverage and greater bug-finding effectiveness.
- *Significance*. Our approach focuses on detecting *soundness bugs*, which are highly impactful. Using Octopus, we discovered 13 new bugs in Z3, cvc5, and Bitwuzla, with nine already fixed by the developers.

Article Organization. The rest is structured as follows. In Section 2, we provide a concrete example to illustrate the high-level idea of our approach. Section 3 formalizes our approach and describes

the implementation of Octopus. We then present a comprehensive evaluation of our approach in Section 5, followed by a detailed discussion of the obtained results in Section 6. In Section 7, we discuss the threats to validity. Subsequently, we survey related work in Section 8, and finally, we conclude our findings in Section 9.

2 Background and Motivation

This section first presents a brief introduction to SMT solvers and their input languages. Particularly, we present the SMT-LIB language and the standard for describing SMT formulas in detail. Then, we motivate our technique using real examples.

2.1 Background

SMT Solver and SMT-LIB Standard. SMTs extend the Boolean satisfiability problem by allowing the decision of satisfiability for formulas involving various first-order logic theories, such as integers, reals, arrays, and strings. A logic formula typically consists of multiple atomic formulas that do not contain further sub-formulas. These atomic formulas or sub-formulas can be combined using logical connectives, such as conjunction and disjunction, to form complex formulas. An SMT solver attempts to solve these intricate formulas and provides a determination of their satisfiability based on its capability. If the solver cannot make a determination, it may yield an "unknown" result.

Various SMT solvers can employ different input formats, e.g., SMT-LIB standard language [6] and BTOR2 [31]. Among these formats, the SMT-LIB language [6] is the most commonly adopted and supported by most of the mainstream solvers. The most basic commands in the SMT-LIB language are `declare-fun`, `assert`, `check-sat`, and `get-model`. The `declare-fun` command is used to introduce new symbols, allowing for the declaration of variables and functions in a unified way. Variables are treated as functions without arguments in this language. For example, the statement `(declare-fun x () Int)` declares the variable x as an integer type. The `assert` command adds constraints to the variables that the formula should satisfy. For example, the statement `(assert (and (< x 3) (> y 1)))` asserts that variable x should be less than 3 and variable y should be greater than 1. The `check-sat` statement is employed to query the solver to determine the satisfiability of a formula. Typically, an SMT instance consists of multiple `assert` statements. Therefore, the definitions of satisfiable and unsatisfiable formulas are as follows.

Definition 2.1 (*Satisfiable Formula*). A formula φ is satisfiable if there exists at least one assignment of values to the variables in φ that makes φ true.

Definition 2.2 (*Unsatisfiable Formula*). A formula φ is unsatisfiable if there is no assignment of values to the variables in φ that makes φ true.

2.2 Motivating Example

Here, we motivate our approach by comparing it with the SOTA SMT solver fuzzer, STORM, using a concrete example. Listing 2 presents an SMT formula written in the SMT-LIB language, representing the logical expression "$(a = \texttt{true}) \wedge ((x > 4) \vee (x < 3 \wedge\ y > 1))$." The formula φ consists of two assertions: the first asserts that the Boolean variable a is `true`, while the second asserts that x and y satisfy the expression "$(x > 4) \vee ((x < 3) \wedge (y > 1))$."

STORM begins by feeding the formula φ to an SMT solver to check its satisfiability. If φ is satisfiable, the solver returns `sat` along with an assignment, such as $\mathcal{A}$: $a := \texttt{true} \wedge x := 5 \wedge y := 0$, which satisfies both assertions. Next, STORM recursively extracts sub-formulas from φ, resulting in six sub-formulas as listed in Listing 3. Among these sub-formulas, φ_5, φ_4, φ_2, and φ_3 are also sub-formulas of φ_6, while φ_2 and φ_3 are sub-formulas of φ_4. STORM evaluates the truth values

```
(declare-fun a () Bool)
(declare-fun x () Int)
(declare-fun y () Int)
(assert (= a true))
(assert (or (> x 4)
        (and (< x 3) (> y 1))))
(check-sat)
(get-model)
```

Listing 2. An SMT formula φ.

```
φ1: (= a true), φ2: (< x 3), φ3: (> y 1)
φ4: (and (< x 3) (> y 1)),φ5: (> x 4)
φ6: (or (> x 4) (and (< x 3) (> y 1)))
```

Listing 3. The sub-formulas extracted from φ in Listing 2.

```
(declare-fun a () Bool)
(declare-fun x () Int)
(declare-fun y () Int)
(assert (= a true))
(assert (and (< x 3) (> y 1)))
(check-sat)
```

Listing 4. A simplified formula φ' revealing a soundness bug.

of these sub-formulas under the assignment $\mathcal{A}$, as displayed in Table 1. Concretely, φ_1, φ_6, and φ_5 evaluate to `true`, while the remaining sub-formulas evaluate to `false`. Utilizing the extracted sub-formulas and their truth values, STORM proceeds to synthesize new formulas to test the SMT solver. If a synthesized new formula φ_n evaluates to `true` under the assignment $\mathcal{A}$, it indicates that $\mathcal{A}$ satisfies all constraints within φ_n. Finally, φ_n can be directly passed to test the SMT solver, and a bug is detected if the SMT solver returns `unsat` for the satisfiable formula φ_n.

However, STORM lacks flexibility in generating diverse test formulas. Specifically, when a formula evaluates to `false`, STORM cannot use it directly for testing, as it cannot determine if there exists an alternative assignment that satisfies the formula. For example, consider the formula φ' in Listing 4, which comprises sub-formulas φ_1 and φ_4 from Listing 3. φ' evaluates to `false` under the assignment $\mathcal{A}$, making it impossible to infer its satisfiability based solely on this assignment. However, φ' is indeed satisfiable under a different assignment, such as $\mathcal{A}'$: $a := \texttt{true} \wedge x := 2 \wedge y := 2$, where φ' evaluates to `true`. The formula is derived from a test case generated by our approach Octopus, exposing a soundness bug in Z3,[3] which incorrectly classifies it as unsatisfiable. STORM, however, is incapable of generating such test instances with known satisfiability solely based on the truth value under $\mathcal{A}$. Furthermore, due to the deterministic nature of solvers [11], they typically produce the same assignment for a given formula. Consequently, STORM's inability to diversify its test cases hinders its effectiveness in detecting this bug.

In general, each sub-formula extracted from the original formula evaluates to a fixed truth value, either `true` or `false`, under a specific assignment. This limitation restricts the diversity of test formulas generated by STORM. However, if multiple assignments are obtained for a formula, its sub-formulas can evaluate to different truth values, allowing for more flexible construction of test formulas. This leads to more comprehensive testing and enhances the potential for discovering bugs. Based on this observation, we propose our approach, which leverages multiple assignments to guide the generation of more diversified test formulas. Our approach will be explained in greater detail in Section 3.

3 Approach

This section introduces Octopus, as depicted in Figure 1, and outlines its key components and phases. Octopus consists of four main phases: sub-formula extraction, multiple assignment collection, formula snippet generation, and instance construction. Algorithm 1 provides a comprehensive description of Octopus's implementation. We delve into each phase's unique design in the following

[3] https://github.com/Z3Prover/z3/issues/5456.

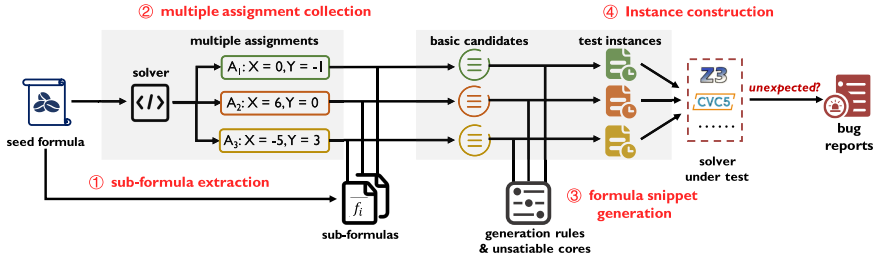


Fig. 1. An overview of Octopus.

subsections, highlighting Octopus's distinctive innovative components, i.e., *assignment collection* and *formula generation*. Finally, we provide formal descriptions with proof of our approach.

3.1 Sub-Formulas Extraction

In general, Octopus employs a method similar to STORM for extracting sub-formulas from a seed formula. For completeness, we include a brief explanation of this sub-formula extraction process. A first-order logic formula is typically composed of multiple nested sub-formulas connected by logical connectives, such as "∧" and "∨." By treating logical connectives as non-leaf nodes and atomic formulas (i.e., those without connectives) as leaf nodes, the formula can be represented as a tree. Note that Octopus does not further decompose these atomic formulas, which are all the Boolean types. Consequently, all extracted sub-formulas are also of Boolean type. The depth of this tree reflects the structural complexity of the formula to some extent. Algorithm 1 demonstrates that the *Fuzz* function initially extracts sub-formulas from the seed formula, denoted as s. This process involves collecting all sub-formulas from the seed, including the seed formula itself and its sub-formulas, and adding them to the initial pool P_{init} (Line 3). For example, assume that the formula φ shown in Listing 2 is the seed formula. Octopus will extract all sub-formulas of the formula φ. These extracted sub-formulas, depicted in Listing 3, serve as the fundamental building blocks for synthesizing new test formulas. The synthesis process will be elaborated upon in the following subsections.

Example. Consider the formula φ shown in Listing 2 as the seed formula. Octopus extracts all sub-formulas from φ, which are illustrated in Listing 3. Concretely, it contains six sub-formulas in total, including `(= a true)`, `(or (> x 4) (and (< x 3) (> y 1)))`, `(> x 4)`, `(and (< x 3) (> y 1))`, `(< x 3)`, and `(> y 1)`. These sub-formulas act as fundamental building blocks for synthesizing new test formulas, which will be elaborated in the following steps.

3.2 Multiple Assignments Collection

The availability of assignments that evaluate sub-formulas to different truth values allows us to generate a broader range of test formulas with known satisfiability. However, due to the complexity of first-order logic formulas and the interdependencies among sub-formulas, achieving comprehensive assignments that cover all possible combinations is impractical in most cases. Therefore, our objective in this step is to cover a broader range of truth values for the sub-formulas, enabling the flexible construction of diverse formulas with test oracles.

To begin, Octopus feeds a seed formula s to an SMT solver. If the SMT solver determines that s is satisfiable, Octopus obtains multiple assignments for s, which are stored in M (Line 5). In case where s is unsatisfiable, Octopus first negates s to obtain a negated formula $\neg s$ and employs the SMT solver to find multiple assignments for $\neg s$ (Line 8). In Algorithm 2, we present the iterative process of assignment collection. The main body of the process follows the common practice of obtaining

Algorithm 1: Main Procedure of Octopus

```
Data: SMT solver T, seed formula s, iteration limit iteration
1  Procedure Fuzz(s, T, iteration)
2      Bugs ← EmptyList()
3      P_init ← ExtractSubFormula(s)                    // Extract sub-formulas from seed
4      if isSat(s) then
5          M ← GetMultiAssign(s)
6      else
7          C ← GetUnsatCore(s)
8          M ← GetMultiAssign(¬s)
9      foreach m ∈ M do
10         P_gen ← GenerateFormula(m)                   // Generate formula snippets from assignments
11         P_temp ← Evaluate(P_init, m)
12         P_constr ← EmptyPool()
13         for i ← 1 to iteration do
14             P_constr ← Assembly(P_temp, P_constr, P_gen)
15         foreach f, v ∈ P_constr do
16             if v = False then
17                 f ← NOT(f);   v ← ¬v
18         if Exist(C) then
19             Oracle ← RandSelect(SAT, UNSAT)
20         else
21             Oracle ← SAT
22         I ← GenerateInstance(P_constr, Oracle, C)
23     foreach inst ∈ I do
24         r ← T.CheckSat(inst)
25         if r ≠ Oracle then
26             Bugs ← Append(T, Bugs, inst)
27     return Bugs
28 Function GenerateInstance(P, Oracle, C)
29     cand ← RandSelect(P, N_assertion)
30     if Oracle = SAT then
31         I ← Instantiate(cand)
32     else
33         I ← Instantiate(cand ∧ C)
34     return I
```

a single assignment. Particularly, when an assignment is obtained, we incorporate additional constraints into the original formula based on the assignment to obtain multiple assignments (Line 6 in Algorithm 2). These constraints ensure that the solver can produce an assignment that differs from the ones already obtained for the seed formula. For example, if the assignment "$\mathcal{A}$: $a := \texttt{true} \wedge x := 5 \wedge y := 0$" is obtained for the formula φ in Listing 2, additional constraints such as "$a \neq \texttt{true} \vee x \neq 5 \vee y \neq 0$" are added to the original formula φ to derive another assignment. Without modifying the constraints in this way, solvers tend to consistently return the same assignment for a given seed formula [11].

Algorithm 2: Obtain Multiple Assignments

```
1 Function GetMultiAssign(constraint)
2     model ← ∅
3     for i = 1 to n_model do                                   ▷ n_model = 10
4         if isSat(constraint) then
5             m_i ← GetAssign(constraint) model ← model ∪ m_i
6             constraint ← constraint ∧ ¬m_i
7         else
8             return model
9     return model
```

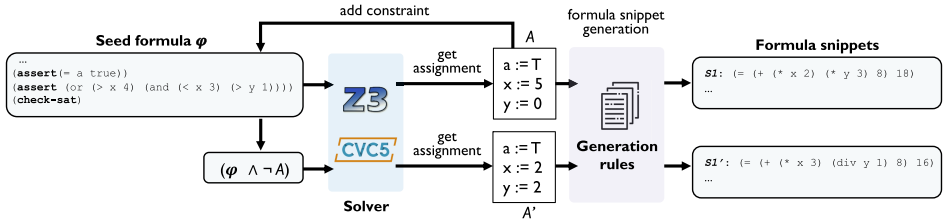


Fig. 2. Multiple assignment collection and formula snippet generation.

As demonstrated in Table 1, more than half of the sub-formulas can exhibit divergent truth values under the assignments $\mathcal{A}$ and $\mathcal{A}'$. Consequently, this approach enables the acquisition of multiple assignments that lead to different truth values for the sub-formulas, thereby facilitating the generation of diverse test formulas. In practice, after obtaining an assignment, we utilize it to evaluate the truth values of all the sub-formulas within the original formula. In particular, this can be efficiently accomplished by substituting the variables in each sub-formula with the corresponding concrete values from the assignment. If the truth value of any sub-formula under the current assignment differs from its truth value under any previous assignment, we add the current assignment to M. This implies that incorporating the assignment will inevitably lead to divergent truth values for some sub-formulas within the original formula.

Through this straightforward yet effective method, we are able to obtain multiple assignments for the seed formulas. These assignments are then used to guide the construction of diverse test formulas, each with known satisfiability. This design differentiates Octopus from the existing technique STORM and enhances its bug-finding capability. Additionally, these assignments can be leveraged to generate viable formula snippets in subsequent steps, enriching the set of basic building blocks for formula construction.

Example. As shown in Figure 2, Octopus derives multiple assignments using an SMT solver. For instance, Octopus may initially obtain the assignment "$\mathcal{A}$: $a := \texttt{true} \wedge x := 5 \wedge y := 0$" by invoking a solver to solve the seed formula φ. Subsequently, Octopus introduces a new constraint that compels the solver to find an assignment distinct from $\mathcal{A}$. This is achieved by conjoining the original constraint with the negation of $\mathcal{A}$, i.e., "$a \neq \texttt{true} \vee x \neq 5 \vee y \neq 0$." The solver will then generate a different assignment, such as "$\mathcal{A}'$: $a := \texttt{true} \wedge x := 2 \wedge y := 2$." This process can be iteratively applied to obtain additional assignments until we cannot collect or reach the upper bound we set.

Algorithm 3: *Formula Generation* Based on Assignment

```
1 Function GenerateFormula(m, P)
2     repeat
3         v1, v2 ← SelectVar(m)
4         skeleton ← GetSkeleton(v1, v2)
5         operators ← SelectOp(skeleton)
6         f, v ← Generate(v1, v2, operators, m, skeleton)
7     until f ∉ P
8     return f, v
```

Table 2. Generation Rules Categorized by Theory Int, Real, String, Bit-Vec, and Floating Point

Theory	Logic	Formula Skeleton	Operation		
			op_1	op_2	op_3
Int	linear	$x\ op_1\ c_1\ op_2\ y\ op_1\ c_2\ op_2\ c_3\ op_3\ R$	$*, div, mod$	$+, -$	$=, \geqslant, \leqslant$
	non-linear	$c\ op_1\ x\ op_1\ y\ op_3\ R$	$*, div, mod$		$=, \geqslant, \leqslant$
Real	linear	$x\ op_1\ c_1\ op_2\ y\ op_1\ c_2\ op_2\ c_3\ op_3\ R$	$*, /$	$+, -$	$=, \geqslant, \leqslant$
	non-linear	$c\ op_1\ x\ op_1\ y\ op_3\ R$	$*, /$		$=, \geqslant, \leqslant$
String		$x\ str.++\ c\ str.++\ y\ op_3\ R$			$=, str.contains$
Bit-Vec		$x\ op_1\ y = R$	$bvsmul, bvsdiv, bvsand$		
		$x\ op_1\ c\ op_1\ y = R$	$bvadd, bvsub$		
FP		$x\ op_1\ y = R$	$fp.add, fp.sub, fp.mul, fp.div$		

c_1, c_2, and c_3 are randomly selected constants. R is the result calculated according to the assignment and random constants.

3.3 Formula Snippets Generation

While leveraging multiple assignments from a seed formula allows generating numerous test formulas with known satisfiability as discussed previously, relying exclusively on sub-formulas present in the original seed can limit the structural diversity of these tests. To overcome this limitation, Octopus incorporates a mechanism to generate novel formula snippets based on the available variable assignments. These snippets, which may not appear in the original seed, serve as additional building blocks, thereby enriching the variety of candidate sub-formulas used during test construction. The generation process, detailed in Algorithm 3, employs a set of predefined rule templates, referred to as *skeletons*, specific to different SMT logics, as shown in Table 2. These rules are designed to cover fundamental operations defined in the SMT-LIB standard across various theories (e.g., integers, reals, strings, bit-vectors, floats), ensuring broad compatibility with SMT solvers and capturing diverse semantic features. Crucially, each generated snippet is constructed such that it definitely evaluates to `true` under the specific variable assignment used for its generation.

Generation begins by randomly selecting one or more variables from the current assignment (Line 3). The choice of the generation rule depends on the data types (i.e., sorts) of the selected variables. For instance, when dealing with integer variables (x, y), a skeleton like $c_1 * x + c_2 * y + c_3 = R$ might be chosen. Here, c_1, c_2, c_3 are randomly generated integer constants. The value R is then calculated based on the current assignment values of x and y and the chosen constants c_i, ensuring the resulting formula (e.g., $1 * x + 1 * y + 1 = 6$ if $x = 2, y = 3$) is true under that assignment.

Similar procedures apply non-linear operations for non-linear integer logic and analogous rules for real arithmetic. For string variables, snippets might involve concatenating variables with random constant strings c, or using predicates like "str.contains". In the latter case, the result R might be determined by randomly selecting the value of one of the involved variables under the current assignment. For fixed-size bit-vectors and floating-point numbers, rules require variables of the same sort; random constants c of the appropriate sort are generated, and operations are applied similarly to ensure the resulting snippet evaluates to true.

These generated snippets, whose structure and constants vary depending on the random choices made and the specific assignment used, are collected into a pool (P_{gen}) (Line 10). They are subsequently used alongside the sub-formulas extracted from the original seed as candidate components for constructing new, complex test formulas. By introducing these novel, assignment-derived snippets, Octopus significantly increases the diversity of the generated test cases compared to methods relying solely on the initial seed's sub-formulas.

Example. Using the obtained assignments, Octopus generates formula snippets according to the rules specified in Table 2. For example, Figure 2 shows snippets generated from two assignments, $\mathcal{A}$ and $\mathcal{A}'$. Consider assignment $\mathcal{A}$, where $a :=$ `true`, $x := 5$, and $y := 0$. Octopus first randomly selects a generation rule applicable to the logic, in this case, quantifier-free linear integer arithmetic. Suppose the selected rule is the first one in Table 2, represented as the skeleton: $x\ op_1\ c_1\ op_2\ y\ op_1\ c_2\ op_2\ c_3\ op_3\ R$. Next, Octopus fills in the operator placeholders (e.g., op_1, op_2) using the operator choices defined in the table. This substitution yields an updated skeleton, for example: $x * c_1 + y * c_2 + c_3 = R$. Octopus then instantiates the constant placeholders (c_1, c_2, c_3) with randomly generated values. Assume the instantiated skeleton is: $x * 2 + y * 3 + 8 = R$. To ensure the formula evaluates to `true` under assignment $\mathcal{A}$, we determine R by substituting $x = 5$ and $y = 0$, yielding $R = 18$. The final formula is: $x * 2 + y * 3 + 8 = 18$, which in SMT-LIB format is: `(= (+ (* x 2) (* y 3) 8) 18)`.

Notably, this generated formula is not equivalent to any sub-formula in the original seed and can thus be added to the construction pool to increase the diversity of test formulas. Similar steps can be applied to other assignments to generate additional snippets, each guaranteed to be evaluate to `true` under their corresponding assignments.

3.4 Instances Construction

Unsat Core Utilization. To support the generation of unsatisfiable formulas, Octopus can incorporate an *unsatisfiable core* when the seed formula is unsatisfiable. An unsatisfiable core refers to a minimal subset of the original formula that is itself unsatisfiable. Specifically, Octopus retrieves the unsatisfiable core from an SMT solver using the `(get-unsat-core)` command defined in the SMT-LIB standard (Line 7 in Algorithm 1). By integrating this core into the constructed formulas, Octopus can deliberately produce unsatisfiable formulas, expanding beyond STORM's capability of generating only satisfiable ones. This extends beyond the capabilities of other solution-aided reassembling approaches, which generate only satisfiable formulas.

Formula Assembling. The process of constructing test instances in Octopus builds on the approach used by STORM, while introducing additional flexibility. In particular, test instances must be valid inputs for SMT solvers, probably consisting of multiple assertions written with the `(assert)` command, along with control commands such as `(check-sat)` to trigger satisfiability checks. To prepare the components for assembling, Octopus first maps each extracted sub-formula to its evaluated truth value under the current assignment. These sub-formulas, together with their truth values, are collected into a temporary pool P_{temp} (Line 11). Octopus then constructs new formulas by randomly selecting logical connectives—specifically, $\wedge$ and $\neg$, which form a functionally complete set [28]—to combine formulas (Algorithm 4, Line 3). For *AND* operations (Lines 4–7),

Algorithm 4: *Formula Construction*

```
1  Function Assembly(P_temp, P_constr, P_gen)
2      f_1, v_1 ←RandSelect(P_temp, P_constr, P_gen)
3      op ← RandSelect(AND, NOT)
4      if op = AND then
5          f_2, v_2 ←RandSelect(P_temp, P_constr, P_gen)
6          f ← AND(f_1, f_2)
7          v ← v_1 ∧ v_2
8      else
9          f ← NOT(f_1)
10         v ← ¬v_1
11     if Depth(f) ≤ D_max then                          ▷ D_max = 32
12         P_constr ← Add(P_constr, f, v)
13         return P_constr
```

formulas are conjoined; for *NOT* operations (Lines 8–10), a formula is negated. The operands are sampled from pools containing sub-formula candidates, intermediate snippets, and previously constructed formulas. The truth value of each new formula is computed based on its sub-formulas, and the resulting formula is added to the constructed pool P_{constr} alongside its truth value.

Before generating test instances, any formula in P_{constr} with a truth value of *false* is negated (Lines 15–17 in Algorithm 1), ensuring that constructed instances are satisfiable by default. If an unsatisfiable core is available, Octopus randomly determines whether to generate a satisfiable or unsatisfiable instance (Lines 18–19); otherwise, only satisfiable instances are produced. This design enables Octopus to explore both satisfiable and unsatisfiable paths, increasing its ability to detect solver soundness errors. Finally, Octopus selects a random subset of formulas from P_{constr} to serve as assertions in the test instance (Line 22). If generating an unsatisfiable instance, the unsatisfiable core is conjoined with formulas from P_{constr} to preserve unsatisfiability (Lines 30–33). The constructed instances are then submitted to the target solver, and any instance yielding a result inconsistent with the oracle is identified as a potential soundness bug (Line 24). These bug-triggering instances are saved for further analysis.

Example. In this step, Octopus combines the extracted sub-formulas from the original seed formula φ with additional formula snippets to generate test instances. Specifically, from φ, we extract sub-formulas $\{\varphi_1, \varphi_6, ..., \varphi_3\}$, and we generate formula snippets $\{s_1, s_2, ...\}$ under an assignment $\mathcal{A}$. All generated snippets are constructed to evaluate to `true` under $\mathcal{A}$. We also record the truth value of each sub-formula under this assignment, for example, $\{\varphi_1$: T, φ_6: T, ..., φ_3: F$\}$. Next, Octopus mutates these formulas using logical connectives, such as conjunction and negation, to produce a diverse pool of new formulas, for example, $\{\varphi_1 \wedge \varphi_6$: T, $\varphi_6 \wedge s_1$: T, $\neg\varphi_5$: F, ...$\}$. From this pool, Octopus selects combinations of formulas that are guaranteed to evaluate to `true` under $\mathcal{A}$ and assembles them into complete test instances, adding necessary SMT-LIB commands such as `(check-sat)`. Notably, if an unsatisfiable core is available, Octopus can also construct unsatisfiable test cases by incorporating it. Through this assembly process, Octopus generates complex and diverse test formulas with known satisfiability, enabling the exercise of SMT solvers, and any discrepancies in solver results are reported as potential bugs.

3.5 Formalization of Octopus

Building upon the design of Octopus outlined previously, we now formalize its principal strategies.

Formal Description of Octopus. Let φ be a seed formula. We define $\mathcal{A}_{\text{set}} = \{\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_k\}$ as a set of k distinct assignments, where each assignment $\mathcal{A}_j \in \mathcal{A}_{\text{set}}$ maps the variables in φ to values from their respective domains (e.g., Booleans {true, false}, integers $\mathbb{Z}$, reals $\mathbb{R}$). Let $\text{Sub}(\varphi) = \{\varphi_1, \varphi_6, \ldots, \varphi_n\}$ denotes the set of all sub-formulas of φ. The truth value of a formula (or sub-formula) φ under an assignment $\mathcal{A}_j$ is denoted by $\text{val}(\varphi, \mathcal{A}_j)$. If φ is unsatisfiable, $C(\varphi)$ denotes an unsatisfiable core of φ.

Octopus employs the following key strategies to generate new formulas:

(1) *Multi-Assignment-Guided Formula Synthesis.* Octopus leverages a diverse set of assignments $\mathcal{A}_{\text{set}}$ to identify sub-formulas $\varphi_i \in \text{Sub}(\varphi)$ that exhibit differing truth values:

$$\exists\, \varphi_i \in \text{Sub}(\varphi),\ \exists\, \mathcal{A}_j, \mathcal{A}_k \in \mathcal{A}_{\text{set}} \text{ such that } \text{val}(\varphi_i, \mathcal{A}_j) \neq \text{val}(\varphi_i, \mathcal{A}_k).$$

This enables the flexible synthesis of new formulas by recombining sub-formulas across multiple assignments.

(2) *Formula Snippet Generation.* Octopus generates a set of new formula snippets, denoted $S(\mathcal{A}, \mathcal{R}) = \{s_1, s_2, \ldots, s_m\}$. These snippets are derived from the assignments $\mathcal{A}$ and predefined rules $\mathcal{R}$ and are explicitly distinct from the existing sub-formulas of φ:

$$\forall\, s_p \in S(\mathcal{A}, \mathcal{R}),\ s_p \notin \text{Sub}(\varphi).$$

These novel snippets expand the pool of building blocks available for constructing test formulas, thereby enhancing diversity.

(3) *Leveraging Unsatisfiable Cores.* When the seed formula φ is unsatisfiable, Octopus extracts an unsatisfiable core $C(\varphi)$. The tool can optionally generate unsatisfiable formulas by including $C(\varphi)$.

In summary, Octopus generates a diverse set of test formulas $\mathbf{T}_{\text{Octopus}}$ according to:

$$\mathbf{T}_{\text{Octopus}} = \mathcal{F}(\text{Sub}(\varphi), \mathcal{A}, S(\mathcal{A}, \mathcal{R}), C(\varphi)),$$

where $\mathcal{F}$ denotes the synthesis function that combines these components: the seed's sub-formulas, the newly generated snippets, and the unsatisfiable core when applicable. This method generalizes prior solution-aided reassembly approaches (e.g., STORM), which often rely on a single assignment:

$$\mathbf{T}_{\text{SAR}} = \mathcal{F}(\text{Sub}(\varphi), \{\mathcal{A}_0\}).$$

Correctness Guarantees. We formally establish that the formulas constructed by Octopus preserve the expected satisfiability properties.

Proposition 3.1. *Let φ be an SMT formula with sub-formulas* $\text{Sub}(\varphi)$*, and let $\mathcal{A}$ be any assignment to the variables in φ. Any formula φ' composed solely of these sub-formulas (or a subset thereof) is satisfiable if it evaluates to true under $\mathcal{A}$.*

Proof. Since $\mathcal{A}$ satisfies φ', it follows directly from Definition 2.1 that φ' is satisfiable. □

Proposition 3.2. *Let φ be an unsatisfiable formula, and let $C(\varphi)$ be its unsatisfiable core. Then, any formula formed by conjoining $C(\varphi)$ with any other formula φ' (i.e., , $\varphi' \wedge C(\varphi)$) is unsatisfiable.*

Proof. Since $C(\varphi)$ is unsatisfiable, no assignment can satisfy it. If $\varphi' \wedge C(\varphi)$ were satisfiable, it would imply the existence of an assignment satisfying both φ' and $C(\varphi)$, contradicting Definition 2.2. Hence, $\varphi' \wedge C(\varphi)$ is unsatisfiable. □

Propositions 3.1 and 3.2 formally establish that Octopus's formula construction strategies correctly produce satisfiable and unsatisfiable formulas as intended. These theoretical guarantees

lay a rigorous foundation for using Octopus as an oracle-guided fuzzing tool to evaluate SMT solver correctness.

4 Implementation

We implemented Octopus using 2,142 lines of Python code. Several key aspects of our implementation are detailed below.

First, to prevent SMT solvers from encountering difficulties due to the complexity of the generated instances, we adopted an approach similar to prior work [28], imposing constraints on the size of the instances from three perspectives. Specifically, we limited the number of construction iterations, the depth of formulas within a single assertion, and the number of assertions in an instance. Concretely, we set the iteration limit (i.e., *iteration* in Line 13, Algorithm 1) to 500, capped the number of assertions per instance (i.e., $N_{assertion}$ in Line 29, Algorithm 1) at 32, and ensured that the depth of each formula (i.e., D_{max} in Line 11 Algorithm 4) did not exceed 32. Note that we set these limitations referring to the prior work [28], which has thoroughly investigated their influence. By adhering to these constraints, we generate instances of an appropriate size, thereby improving testing throughput.

Second, given that an SMT formula can contain numerous sub-formulas, the seed formula may have multiple assignments that evaluate the sub-formulas to different truth values. To maintain efficiency and maximize testing throughput, it is necessary to impose limits in this scenario. We introduced a cap on the number of assignments generated for each seed and imposed a timeout for solving the seeds. Typically, the number of assignments does not exceed 10, and the timeout for solving a sub-formula is set to 5 seconds.

Third, the number of new instances generated for each seed is contingent upon the number of assignments obtained for that seed. The more assignments we obtain, the greater the number of test instances generated. Specifically, for each assignment derived from a given seed, we generate 500 test instances to use in the fuzzing process.

5 Evaluation

In this section, we present extensive evaluations aimed at investigating the effectiveness of our tool. Specifically, the experiments aim to answer the following **Research Questions (RQs)**:

— *RQ1: (Bug Detection). Can Octopus detect new critical bugs in SMT solvers?*
The objective of RQ1 is to determine whether our tool, Octopus, is capable of identifying new, significant bugs in SMT solvers. This criterion is crucial in assessing the effectiveness of fuzzers. The answer to RQ1 provides insights into the contribution of Octopus toward enhancing the reliability of SMT solvers.

— *RQ2: (Capability Comparison). How effective is Octopus compared to the SOTA fuzzers?*
The aim of RQ2 is to evaluate the bug-finding capabilities of Octopus in comparison to existing oracle-guided SMT fuzzers. Our primary intention is to propose a general technique that can effectively validate SMT solvers and complement the existing techniques. The answer to RQ2 helps us understand the effectiveness of Octopus and the advantages brought by our approach, which utilizes multiple assignments to enhance mutant diversity.

— *RQ3: (Sensitivity Analysis). How do the two components contribute to the overall effectiveness of Octopus?*
The objective of RQ3 is to comprehensively investigate the contributions of the major components of Octopus. First, Octopus generates multiple assignments to guide the construction of new test inputs with oracles. Second, to further leverage assignments and improve mutant diversity, Octopus injects formula snippets for participation in mutation. The answer to RQ3

allows us to understand the contributions of these components toward overall performance, including bug detection and improved code coverage.

5.1 Evaluation Setup

In this subsection, we will introduce the overall experimental setup, including the targeted solvers, the test seeds, and the hardware configuration.

Targeted Solvers. We selected `Z3` and `cvc5` for our experimental evaluation, consistent with prior work [12, 28, 32, 47]. Our choice was guided by several key considerations. First, these solvers are widely integrated into prominent frameworks—`Z3`, for example, is enabled in `KLEE`—making them highly influential in both research and practice. Second, they provide robust support for a diverse range of logical theories and have demonstrated strong performance across many problem categories. Third, as mature, actively maintained open source projects, they continue to improve in soundness and reliability through contributions from both developers and researchers. These qualities make `Z3` and `cvc5` not only suitable but also challenging targets for evaluating the effectiveness of our approach. Moreover, the active development communities behind these solvers enable prompt feedback and resolution of reported issues, facilitating a collaborative and impactful evaluation process. To further broaden our study, we additionally considered `Bitwuzla`, a recent high-performing solver, demonstrating the applicability of our methodology across multiple SOTA tools. Similar to prior work [28, 47, 48], we enable some necessary options of solvers for testing. For example, the `-strings-exp` and `-incremental` in cvc5 are enabled to support comprehensive functions in string theory and incremental solving, respectively [47].

Test Seeds. In order to conduct a comprehensive evaluation of Octopus, we chose SMT-LIB instances in SMT official benchmarks [37] as the seed formulas in the experiments. The reason is that the SMT-LIB benchmarks make the largest collection of SMT instances in the SMT-LIB 2.6 language [7]. Those seeds have also been tested by various solvers during the competition, and thus, the solvers can correctly generate the result and model. As mentioned in Section 3.3, we design a series of unique generation rules for the five theories, that is, integer, real, string, bit-vector, and floating. In order to better verify the effect of *formula generation*, we focus on the logics that combine those theories to evaluate Octopus. In addition to the seeds in the benchmarks, the SMT instances in solvers' regression test suites are also valuable. Consequently, we take the seeds in the regression test suites of Z3,[4] cvc5,[5] and Yices2[6] into consideration when conducting experiments.

Bug Reporting. To reduce the burden on developers and prioritize the most critical issues, we focused primarily on identifying and reporting soundness bugs, which are generally considered the most severe. To assist in debugging, we minimized the size of each bug-triggering formula using `ddSMT` [24], a delta debugger tailored for the SMT-LIB language. To avoid redundant issue reports, we adopted a conservative reporting strategy. When multiple formulas within the same logic revealed soundness bug, we initially reported only one instance. The remaining instances were withheld and revisited after the original bug was fixed. If additional formulas continued to trigger bugs, we reported them in follow-up issues.

Experimental Environment. We conducted our empirical evaluations on a machine equipped with an Intel Xeon Gold 6240 CPU (16 cores) and 32 GB of RAM, running Ubuntu 18.04 (64-bit). For each seed, we allocated a time budget of 15 minutes, with each solver invocation limited to a maximum of 10 seconds. This setup adheres to the established practices in prior work [23, 28, 54].

[4] https://github.com/Z3Prover/z3test.
[5] https://github.com/cvc5/cvc5/tree/main/test/regress.
[6] https://github.com/SRI-CSL/yices2/tree/master/tests.

Table 3. The Bugs Reported by Octopus in Z3 and cvc5

Solver	#Bug ID	Logic	Type	Status	PK?
Z3	#5373	QF_NRA	Soundness	Fixed	✓
	#5447	QF_ABV	Soundness	Fixed	✗
	#5456	QF_IDL	Soundness	Fixed	✗
	#5457	QF_BV	Soundness	Fixed	✗
	#5460	QF_BV	Soundness	Fixed	✗
	#5468	AUFLIRA	Soundness	Fixed	✗
	#5488	QF_BV	Soundness	Fixed	✗
	#5508	QF_NIA	Soundness	Won't fix	-
	#7027	ALIA	Soundness	Reported	-
	#7217	ABV	Soundness	Confirmed	✗
cvc5	#10057	ABV	Soundness	Fixed	✗
	#10067	NRA	Crash	Confirmed	✗
Bitwuzla	#110	QF_BV	Soundness	Fixed	✗

PK indicates whether the bug is previously known.

5.2 RQ1: Bug Detection

Experiment Setup. In this experiment, we consistently employ Octopus on the latest trunk versions of the targeted solvers to discover new bugs, aligning with the approach adopted in previous studies [28, 40]. Additionally, we also include the solver Bitwuzla to assess the generality of Octopus. Bitwuzla is a relatively new, advanced SMT solver. Although it only supports a narrow range of logics, "*Bitwuzla is a particularly good candidate since it is very robust*" as recommended by one of the cvc5 developers [39].

Results. Table 3 presents the identified bugs reported by Octopus in the solvers. In total, we discovered 13 bugs across nine distinct logics, with 11 bugs confirmed by the developers, of which nine bugs have already been fixed. Notably, we found that 10 of the reported bugs were previously unknown, and the developers promptly addressed the majority of them after receiving the reports. Two issues remain without confirmation or resolution. For issue #5508, the developers explained that it relates to longstanding challenges in non-linear arithmetic and is likely to be addressed only after resolving earlier reports in this category; we therefore classify it as "Won't fix" for now. Issue #7027 was still under investigation at the time of submission.

Furthermore, in addition to fixing the identified bugs, the developers also made further improvements to Z3's rewrite rules, drawing inspiration from the instances we reported (e.g., issue #5457). It is essential to highlight that issue #10057 in cvc5 pertains to a solution soundness bug, which implies that cvc5 incorrectly returns *sat* for an unsatisfiable instance. This type of bug cannot be detected by STORM or Diver in essence, as they cannot generate unsatisfiable instances. In summary, the outcomes of this experiment demonstrate the promising potential of Octopus in uncovering critical soundness bugs within widely used SMT solvers. In Section 6, we will

delve into a more comprehensive discussion regarding the significance of the bugs identified by Octopus.

5.3 RQ2: Capability Comparison

In this RQ, we aim to investigate the capability of different SMT solver testing tools in terms of bug discovery and achieved code coverage. Specifically, we select the two SOTA oracle-guided fuzzers, namely STORM [28], YinYang [48], and Diver [23] as baselines. YinYang and STORM are selected due to their representation of the most advanced oracle-guided approaches. Besides, we include HistFuzz [40] and ET [46] as representative differential testing techniques, which were most recently proposed. All of them are open sourced testing tools.

5.3.1 *Bug Exposure.*

Experiment Setup. To evaluate the effectiveness of our approach, we compared the bug-finding capabilities of Octopus against existing fuzzing tools using historical versions of SMT solvers. Following the experimental protocols used in prior work [39, 40], we conducted our experiments on Z3 version 4.8.5 and CVC4 version 1.7. These versions were chosen for two main reasons. First, many of the bugs previously discovered in these solvers have been addressed in their latest versions, potentially reducing the number of detectable bugs and limiting the ability to assess the effectiveness of fuzzing techniques. Second, using older versions allows us to apply the *Correcting Commit* technique [15], which enables a more objective identification of unique bugs based on solver version history, rather than relying solely on developer feedback or manual analysis. For this experiment, we utilized the instances from the SMT-LIB benchmarks as seeds for the fuzzers. Due to the substantial number of seeds in the benchmarks, we randomly sampled 1,000 formulas from logics supported by all evaluated fuzzers, aligning with prior studies [40, 47]. Each fuzzer then mutated these seed formulas to test the solvers. We executed each testing tool for 24 hours, consistent with the time budgets used in earlier work [39]. After each run, we collected the bug-triggering instances generated by each tool and examined the number of unique correctness bugs revealed by these instances, including refutational soundness bugs and solution soundness bugs. To ascertain whether a bug is truly unique, we utilized the *Correcting Commit* technique [15] to make a distinction. In this process, we ran each bug-triggering formula across multiple versions of the solver to identify the specific commit that fixed the bug. If a formula triggers a bug in earlier versions but not in later ones, we consider the fixing commit to be its correcting commit. Bugs linked to different correcting commits are treated as distinct. To make this process efficient, we used binary search to automatically locate the correcting commit. On average, identifying the correcting commit for a bug takes about thirty minutes. However, when the necessary solver versions are already built, the process can be completed in under two minutes, since the primary cost lies in building the commit versions.

Results. Figure 3 summarizes the results of our evaluation. The results demonstrate that our tool, Octopus, detects more bugs than the baseline fuzzers. Specifically, Octopus detected 8 unique correctness bugs, compared to 3 by STORM and 5 by Diver. Notably, of the 8 bugs found by Octopus, 5 were also detected by at least one of the baselines, while the remaining 3 were uniquely discovered by our tool. These results highlight Octopus's ability to uncover bugs that other tools fail to detect. We attribute this advantage to the use of our *multiple assignments* and *formula generation* techniques, which increase the diversity and effectiveness of generated inputs. Overall, these findings demonstrate that Octopus is a strong complement to existing SMT solver fuzzing techniques, offering improved capabilities in identifying soundness issues in solver implementations.

5.3.2 *Code Coverage.*

Experiment Setup. To further understand the effectiveness of Octopus, we evaluate the code

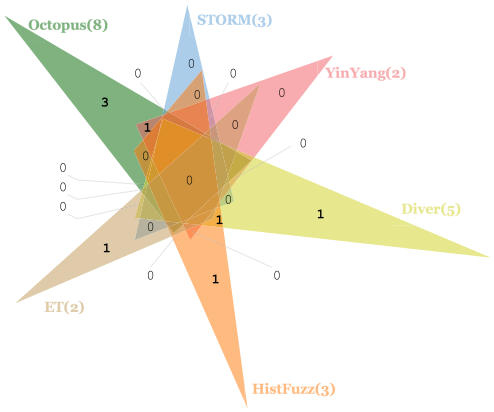


Fig. 3. The distribution of unique correctness bugs detected by Octopus and the baselines.

Table 4. Line Coverage Achieved by Octopus and the Baselines

Tool	Z3	cvc5
YinYang	19.5%	20.1%
STORM	14.4%	18.0%
HistFuzz	29.0%	28.2%
ET	14.7%	14.8%
Diver	14.9%	19.9%
Octopus	26.3%	24.1%

The highest coverage is highlighted with a gray background.

coverage achieved by different fuzzers, including YinYang, STORM, HistFuzz, Diver, ET, and Octopus. Each fuzzer is used to generate test inputs, and the resulting coverage is measured separately. To ensure consistency, we sample 1,000 formulas drawn from SMT-LIB benchmarks with the logics supported by all the evaluated tools. It is worth noting that HistFuzz and ET do not require input formulas as seeds. Following the experimental setup of prior work [48], each tool is executed for 1 hour per trial. We measure the line coverage of the solvers using gcov [2]. All fuzzers are run with their default configurations. For the SMT solvers, we use the latest release versions available at the time of writing: Z3 version 4.14.0 and CVC5 version 1.2.1. To mitigate the influence of randomness, each experiment is conducted 10 times, and the outcomes are averaged over these trials.

Results. Table 4 summarizes the line coverage achieved by Octopus and the baseline tools on Z3 and cvc5. Specifically, Octopus achieves the highest coverage among the oracle-guided techniques, reaching 26.3% on Z3 and 24.1% on cvc5, outperforming YinYang, STORM, ET, and Diver. To assess the statistical significance of these findings, we conducted a Mann-Whitney U-test [27] comparing the coverage results of Octopus with those of the baseline tools. The results indicate that Octopus's superior code coverage over YinYang, STORM, ET, and Diver is statistically significant (p-value $<$ 0.05). Additionally, the differential testing tool HistFuzz attains the highest overall coverage, achieving 29.3% on Z3 and 28.5% on cvc5. This suggests that differential techniques may explore a broader set of code paths. One possible explanation is that HistFuzz starts with a large corpus of formulas collected from previously reported issues, which it uses to synthesize new test cases [40]. Additionally, because HistFuzz does not require generating test oracles, it retains greater flexibility in how it mutates inputs. However, achieving higher code coverage does not necessarily translate to greater effectiveness in detecting soundness bugs. The quality and precision of the test oracle play a crucial role. Therefore, although HistFuzz achieves slightly higher coverage, Octopus finds more soundness bugs, highlighting the advantage of Octopus for uncovering deeper bugs.

Finally, we note that none of the fuzzers achieves more than 30% line coverage on either solver. This relatively low coverage reflects the inherent complexity and scale of SMT solvers; for example, Z3 contains over 450,000 lines of code. Even a small coverage increase, such as 0.1%, can represent hundreds of additional executed lines [48]. These findings underscore the challenges of testing large, highly optimized systems and the practical value of incremental improvements in coverage.

5.4 RQ3: Sensitivity Analysis

We are also curious about the contributions of the two main components in Octopus. To address this RQ, we conducted a comprehensive evaluation of Octopus, focusing on its two major components, obtaining multiple assignments and generating formula snippets. This evaluation also encompassed two aspects: *bug exposure* and *code coverage*. To achieve this goal, we utilized the following variants of Octopus:

- Octopus-MA. This variant employs multiple assignments to guide the construction of new formulas, but does not generate additional formula snippets.
- Octopus-FG. This variant uses only one assignment to guide the construction of new formulas, and generates formula snippets to enhance the construction process.

Experiment Setup. This experiment evaluates the importance of both components of Octopus in relation to its overall efficacy. Specifically, we assess whether each component is necessary by determining if the soundness bugs reported in RQ1 (Section 5.2) can still be detected when either component is removed. We focus on the seven soundness bugs in Z3 that have been confirmed and fixed. To replicate the detection of these bugs, we use Octopus along with two of its variants, Octopus-MA and Octopus-FG, as well as STORM. Specifically, we gather initial seeds that can be mutated to generate bug-triggering instances, along with the corresponding configurations, including the commit of the solver where the bug was discovered. Using this setup, we apply Octopus, its two variants, and STORM to independently mutate each seed. To reduce the influence of randomness, we perform 20 independent runs for each seed-tool pair. If a tool fails to generate a bug-triggering instance in all runs, we consider it unable to detect the bug. If a candidate instance is produced, we test it against the solver's fixed commit to verify whether it corresponds to the reported bug. This process allows us to assess each tool's ability to reproduce the discovered bugs. In addition to bug reproduction, we also evaluate each variant in terms of bug exposure on known issues and overall code coverage. These additional evaluations are conducted following the same experimental procedure as described in RQ2 (Section 5.3).

Results. Table 5 summarizes the results of our bug reproduction experiment. Among the seven previously reported bugs, the Octopus-MA variant is able to reproduce three, while Octopus-FG reproduces four. In comparison, STORM detects only two, highlighting the advantages brought by Octopus. Notably, none of the three tools, Octopus-MA, Octopus-FG, or STORM, is able to detect bugs #5373 and #5457. This suggests that the two components of Octopus work in a complementary manner and that both are necessary for reliably detecting all known bugs. This observation is further supported by the results in Figure 4, which show overlap and differences in the unique known bugs detected by each tool. The data confirm that the full version of Octopus, with both components enabled, is more effective than any single variant. Table 6 shows the corresponding code coverage results, with the highest values highlighted in gray. Specifically, Octopus achieves the highest line coverage, 26.3% for Z3 and 24.1% for cvc5, surpassing both of its ablated variants. Disabling either component leads to a drop in coverage, underscoring the importance of both components in enabling broad exploration of the solver codebase.

6 Discussion

In this section, we provide a detailed discussion of the results obtained from our evaluation.

6.1 Importance of Bug Finding

To further assess the significance of the discovered bugs, we analyzed the release versions of Z3 affected by these previously unknown issues, shedding light on their lifespan. We specifically

Table 5. Bug Reproduction with Different Variants of Octopus and STORM

Variant	#5373	#5447	#5456	#5457	#5460	#5468	#5488
Octopus	✓	✓	✓	✓	✓	✓	✓
Octopus-MA	✗	✓	✗	✗	✓	✗	✓
Octopus-FG	✗	✗	✓	✗	✓	✓	✓
STORM	✗	✗	✗	✗	✓	✗	✓

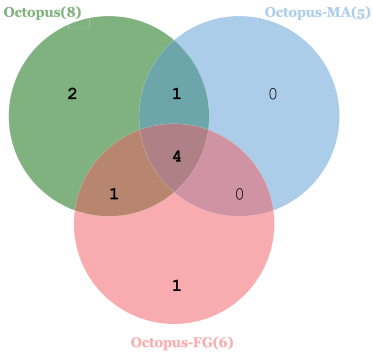


Fig. 4. The distribution of unique correctness bugs detected by Octopus and its variants.

Table 6. Line Coverage Achieved by Octopus and the Baselines

Tool	Z3	cvc5
Octopus-MA	22.4%	19.5%
Octopus-FG	22.7%	20.1%
Octopus	26.3%	24.1%

The highest coverage is highlighted with a gray background.

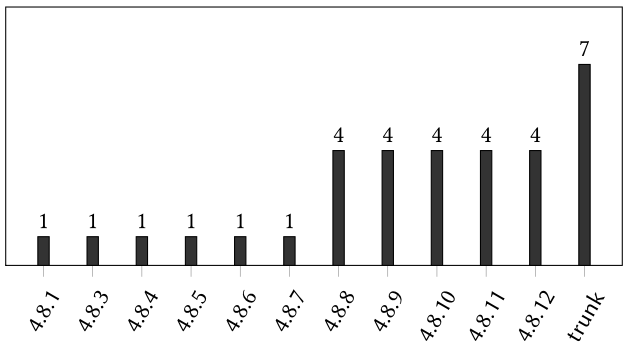


Fig. 5. Fixed bugs reported by Octopus that affect corresponding release versions of Z3.

examined the official releases from version 4.8.1 (October 16, 2018) to 4.8.12 (July 13, 2021). Figure 5 illustrates the impact, revealing that one bug affected Z3 4.8.1, a version released nearly three years prior. This indicates that the bug remained undetected by SOTA techniques for approximately three years. Additionally, we identified four bugs introduced in Z3 4.8.8, which similarly went unnoticed for over a year. These findings demonstrate that Octopus is effective at uncovering long-latent bugs in the SMT solver that were likely overlooked by existing methods.

6.2 Efficiency

To investigate the efficiency of the fuzzers, we examine the number of mutants generated per second. Similarly, we conduct this experiment on Octopus and its two variants aforementioned, as well as the SOTA fuzzers. We ran the experiment ten times, utilizing a single thread on a single core.

Table 7. Number of Mutants Generated per Second

Tool	Avg	Max	Min	StdDev
YinYang	75.7	160.6	22.5	58.2
STORM	89.4	125.6	43.1	29.9
HistFuzz	52.1	54.7	49.8	2.1
Diver	64.3	81.2	36.7	19.7
Octopus-MA	108.9	136.6	66.1	29.1
Octopus-FG	90.8	109.8	28.2	31.1
Octopus	99.5	128.9	51.7	27.7

Table 7 summarizes the results. Notably, we omit ET from this investigation, as it is generation-based rather than mutation-based. It can produce an extremely large number of them within a short timeframe. As shown in Table 7, Octopus generally generates a higher number of mutation files within a short period of time. The three variants of Octopus produce over 90 mutants per second. Notably, the Octopus-MA component exhibits the highest mutation efficiency, reaching 108.9 mutants per second. This can be attributed to the generation of multiple instances from a single seed using multiple assignments, thereby enhancing the efficiency of this component compared to other tools. Although Octopus-FG, the *formula snippets generation* variant, has slightly lower mutation efficiency compared to the other two Octopus variants, it still generates over 90 mutants per second. Overall, as anticipated, our approach demonstrates improved mutation efficiency. This is due to Octopus's ability to utilize multiple assignments and generate more test instances from a seed, thereby enhancing mutation efficiency.

6.3 Bug Sample

To further understand the effectiveness of our approach, we analyze some representative bugs reported by Octopus.

Figure 6 presents a soundness bug in Z3's bit-vector logic, where the solver incorrectly reports unsat for a satisfiable formula. The formula, shown in simplified form after minimization, remains moderately complex, comprising 12 `assert` statements. This bug cannot be detected by STORM because it fails to verify the satisfiability of the first assertion under the solver's provided model. However, Octopus successfully generates the formula with one of the multiple assignments. The root cause of the bug lies in unsafe element access within Z3's internal data structures for bit-vector reasoning. The implementation assumes that index-based lookups into vector-like structures are always valid, without performing bounds checks. This assumption breaks down when comparing structurally different vectors, allowing invalid accesses that silently produce incorrect reasoning outcomes. Z3's developers addressed the issue by introducing safer element retrieval mechanisms that validate indices before access.

Figure 7 illustrates a soundness bug in Z3's simplification pipeline. The input formula asserts that the sum of three bit-vector variables and a constant is not equal to a constant. This formula is clearly satisfiable, yet Z3 incorrectly reports it as unsat. The key structure that triggers the bug was not present in the original seed input but was synthesized by our formula generation rules. This highlights the effectiveness of our approach in discovering novel and critical solver behaviors.

```
(declare-const x_bv (_ BitVec 1))
(declare-const x Bool)
(declare-const x9 Bool)
(declare-const x3 Bool)
(declare-const x4 Bool)
(declare-fun p () (Array (_ BitVec 32) (_ BitVec 8)))

(assert (not (= (_ bv1 32) ((_ zero_extend 24) (select p ...)))))
...
(assert (and (not (and (not x4) (and x9 (not x3)))) ...))
(assert (bvsle (_ bv0 32) (_ bv0 32)))
(check-sat)
```

Fig. 6. Minimized bug-revealing formula for a soundness issue on bit-vector in Z3.

```
(declare-const x Bool)
(declare-fun T () (_ BitVec 32))
(assert (not (= (_ bv1 32) (bvadd T T T (_ bv1 32)))))
(check-sat)
```

Fig. 7. Minimized bug-revealing formula for a soundness issue in Z3's simplification pipeline.

```
(declare-const v (_ BitVec 1))
(declare-fun a () (Array (_ BitVec 8) (_ BitVec 8)))
(assert (and (= (store a (_ bv0 8) (_ bv0 8)) (store a (_ bv0 8) (bvadd (_
bv1 8) ((_ zero_extend 7) v))))))
(check-sat)
```

Fig. 8. Simplified bug-revealing formula for a solution soundness issue in cvc5.

According to Z3's developers, the root cause was an uninitialized expression in a conditional branch of the simplification tactic. This omission caused incorrect intermediate simplifications, ultimately resulting in an unsound outcome.

Figure 8 presents a minimal benchmark that exposes a soundness issue in cvc5. The formula compares two arrays that are identical except for the value stored at the same index. By the semantics of arrays, such arrays can only be equal if the stored values are equal. In this case, however, the stored values can never be equal, which makes the formula unsatisfiable. Concretely, the formula forces equality between storing 0 into the array and storing the result of adding 1 to a zero-extended version of a one-bit variable. The one-bit variable can only take the values 0 or 1. After zero extension, this becomes either 0x00 or 0x01. Adding 1 produces either 0x01 or 0x02. None of these values equal 0x00, so the formula is inherently unsatisfiable. However, cvc5 incorrectly reports `sat`. This inconsistency indicates a solution soundness bug. Existing fuzzing tools such as STORM and Diver cannot reveal this issue because they only generate satisfiable formulas. Our approach, by contrast, naturally produces both satisfiable and unsatisfiable formulas, allowing it to uncover subtle soundness bugs that other methods miss. The cvc5 developers classified this issue as *major*, underscoring its importance.

6.4 Diversity of Mutants

Symbolic Expression Diversity. To assess the syntactic variation introduced by fuzzing, we introduce a *diversity* metric that quantifies structural differences between a generated formula (mutant) and its original seed. The design of this metric is inspired by practices in the field of compiler testing [43]. Particularly, this metric is based on symbolic expressions (s-expressions), the core syntactic units

Table 8. Average Diversity Score Achieved by STORM and Octopus

Tool	Average Diversity Score
STORM	1.40
Octopus-MA	1.80
Octopus-FG	1.81
Octopus	1.83

of the SMT-LIB standard [6], which represent formulas using a nested, parenthesized structure. S-expressions form the building blocks of SMT-LIB formulas. For example, consider the seed formula $F_{\text{seed}} \equiv$ `(= (+ x 5) 10)`. Its constituent s-expressions are: $S(F_{\text{seed}}) = \{$`(+ x 5)`, `10`, `=`, `x`, `+`, `5`$\}$. This set includes six distinct elements.

We define the diversity score of a mutant formula F_{mutant} with respect to its seed F_{seed} as the proportion of s-expressions in the mutant that do not appear in the seed. Formally:

$$D(F_{\text{mutant}}, F_{\text{seed}}) = \frac{|S(F_{\text{mutant}}) \setminus S(F_{\text{seed}})|}{|S(F_{\text{seed}})|}.$$

As an illustration, let $F_{\text{mutant}} \equiv$ `(= (+ x 5) 1)`. Then: $S(F_{\text{mutant}}) = \{$`(+ x 5)`, `1`, `=`, `x`, `+`, `5`$\}$. The mutant introduces one novel s-expression: `1`, which replaces `10` from the seed. Therefore, the diversity score $D(F_{\text{mutant}}, F_{\text{seed}})$ should be $\frac{1}{6}$. This metric provides a simple and interpretable means of assessing how much a fuzzing technique alters the syntactic structure of seed formulas.

Mutant Diversity Analysis. To evaluate the capability of our fuzzing tool, Octopus, to generate diverse mutants, we conducted an analysis comparing it against STORM. The evaluation focused on the diversity metric defined above, measuring the proportion of new s-expressions introduced by each tool into the mutants relative to their corresponding seeds. For this comparison, we utilized the 1,000 seed formulas, consistent with those employed in our other experiments (e.g., Section 5.3). We generated mutants using both Octopus and STORM and calculated their average diversity scores. In addition, we also included the Octopus variants, Octopus-FG and Octopus-MA, as introduced in RQ3 (Section 5.4). Table 8 summarizes the results. Concretely, Octopus achieved an average diversity score of 1.83, while STORM obtained an average score of 1.40. This indicates that Octopus increases the diversity of generated mutants by approximately *31%* compared to STORM. This finding suggests that Octopus is more effective in exploring novel formula structures during the fuzzing process. In particular, both components appear to contribute to increased diversity effectively, especially the *formula generation* component.

6.5 Limitation

Octopus has several limitations. First, Octopus currently employs a limited set of generation rules. While these rules have been effective in practice, expanding them could further improve the diversity of generated test cases. Broadening the range of generation strategies remains an important direction for future work, and Octopus supports the inclusion of user-defined rules to further enhance its ability. In addition, Octopus relies on solver-provided assignments and unsatisfiable cores to construct new inputs. As a result, its performance is inherently tied to the efficiency of the underlying SMT solvers. If an upstream solver operates inefficiently, it can negatively impact the efficiency of Octopus. Nonetheless, as discussed earlier (Section 6.2), the introduced overhead is generally manageable. Finally, Octopus currently focuses on combining formulas using the `and` and `not` operators. Incorporating additional mutation strategies could increase the variety and

complexity of generated test inputs. Exploring these richer mutation techniques represents another promising avenue for enhancement.

7 Threats to Validity

In this section, we examine the threats to the validity of our study. We consider threats to both internal and external validity.

7.1 Internal Validity

Internal validity refers to the degree to which the experimental results are valid for the subjects under test. In this study, the internal threats to validity are as follows.

First, Octopus requires seed instances as input, which raises the possibility of undetected bugs due to a lack of seeds. To mitigate this threat, we carefully selected instances from SMT official benchmarks as seeds, ensuring an adequate number of seeds with diverse semantic features were available. Second, the impact of randomness affects nearly all fuzzers. To alleviate this issue, we repeated the experiments multiple times, as described in Section 5, to minimize the influence of randomness. Third, evaluating effectiveness solely by the number of newly discovered bugs provides only a partial picture. Therefore, in addition to counting new bugs, we also measured the number of known bugs detected and the coverage achieved, offering a more comprehensive assessment of each technique's performance.

7.2 External Validity

External validity refers to the degree to which the experimental results are valid for other subjects. The primary threat to external validity is the choice of the solver under test. In this study, we employed Octopus to discover real bugs in Z3, cvc5, and Bitwuzla, which are representative advanced SMT solvers. While these solvers are critical choices, it is also important to validate our approach with other solvers. As part of future work, we intend to extend our evaluation to include additional SMT solvers. Additionally, the selection of specific versions of the SMT solvers for comparison with Octopus in terms of bug-finding ability may pose a threat to external validity. In our experiments, we used previous versions of the SMT solvers that were released more than two years ago, ensuring a fair comparison. This choice was made because there were few studies focusing on testing SMT solvers during that time, giving the baseline techniques more opportunities to find bugs. Therefore, the external validity of the selected SMT solver releases for evaluating the effectiveness of Octopus in our study can be considered acceptable. Finally, a potential threat arises from the possibility that a single commit may fix multiple bugs, which could obscure the identification of distinct issues when using the Correcting Commit technique. To address this, we manually reviewed all correcting commits identified in our study and confirmed that none included fixes for multiple bugs.

8 Related Work

Testing SMT Solvers. SMT solvers are essential for applications in program analysis, verification, and constraint solving [9, 45, 56]. Ensuring their correctness and robustness has therefore attracted considerable attention [10, 34, 57]. Recent years have seen the emergence of numerous testing techniques for SMT solvers, as shown in Table 9.

Many tools rely on *differential testing*, where multiple solvers are run on the same input and discrepancies indicate potential bugs. Examples include OpFuzz [47], TypeFuzz [32], Murxla [30], ET [46], and HistFuzz [40]. These approaches are effective for detecting functional inconsistencies but face limitations when no reliable reference solver is available. In particular, they cannot expose soundness issues when solver-specific features are involved, or solvers are unable to provide correct

Table 9. Comparison of Octopus with Related SMT Solver Testing Techniques Proposed since 2020

Technique	Sort	Oracle	Bug Types	Generative	Theory
Bugariu and Müller [13]	Mutation	✓	Both	✓	String
STORM [28]	Mutation	✓	Refutational soundness	✗	All
HistFuzz [40]	Mutation	✗	Both	✗	All
YinYang [48]	Mutation	✓	Both	✗	Int, Real, String
OpFuzz [47]	Mutation	✗	Both	✗	All
TypeFuzz [32]	Mutation	✗	Both	✗	Int, Real, String
Diver [23]	Mutation	✓	Refutational soundness	✗	All
Murxla [30]	Generation	✗	Both	✓	All
ET [46]	Generation	✗	Both	✓	Int, Real, BV, FP, String, Array
Octopus	Mutation	✓	Both	✓	All

Oracle: ✓ means the technique can detect soundness bugs with reliable oracle, ✗ means it relies on differential testing. *Generative*: Whether the technique incorporates newly generated sub-formulas rather than only reusing or mutating existing ones.

and definitive outcomes. For instance, both solvers may consistently yield incorrect outcomes, or a reference solver can return unknown. Such unknown cases occur frequently due to the inherent difficulty of SMT problems. In contrast, *oracle-guided* techniques generate inputs with known satisfiability, enabling soundness bug detection without cross-checking multiple solvers. Bugariu and Müller [13] preserve satisfiability while incrementally increasing formula complexity for string solvers. YinYang [48] fuses two formulas into a new satisfiability-preserving one. Other methods such as, STORM [28] and Diver [23], use assignments from seed formulas to guide the creation of satisfiable variants with altered Boolean structures.

Building on this line of work, Octopus is an oracle-guided, mutation-based fuzzer that fully exploits information from seed formulas to generate both satisfiable and unsatisfiable inputs. Unlike most mutation-based approaches, Octopus does more than mutating seed formulas—it also creates new formula components during construction. Compared to STORM and Diver, Octopus is capable of identifying both refutational soundness bugs and solution soundness bugs by leveraging unsatisfiable cores, whereas those tools cannot detect solution soundness bugs by design. In summary, Octopus provides broader validation capabilities across diverse SMT-LIB theories than existing testing techniques, as illustrated in Table 9.

Mutation-Based Fuzzing. Fuzzing is one of the most popular testing techniques due to its good extensibility and ease of deployment. The mutation-based fuzzing technique generates test cases by mutating valid initial seeds. One of the SOTA fuzzers is **American Fuzzy Lop (AFL)** [55]. AFL employs mutations to generate new test cases and improve code coverage through feedback. Subsequently, some grammar-aware fuzzing methods were proposed based on AFL, such as AFLSmart [33] and NAUTULIUS [3]. Furthermore, CodeAlchemist [20] preserves semantic requirements, such as type correlation, as constraints during input generation. For complex software like SMT solvers, it is suitable to employ fuzzing for validation [42]. The methods proposed for testing solvers are generally grammar-aware. However, our approach differs in that it can obtain test oracles from

multiple directions, enabling flexible construction of test formulas and better exploration of SMT solvers.

Metamorphic Testing. Metamorphic testing is a common approach used to address the test oracle problem [17, 35, 41, 44, 50]. It typically involves transforming an existing test input to generate new test cases with known expected outputs. Since its introduction in 1998 [16], metamorphic testing has been applied in various testing domains, including compilers [38], debuggers [52], profilers [51], embedded systems [22, 25], machine learning [29, 49], and web services [58]. Metamorphic testing can also be applied to validate SMT solvers. YinYang [48] synthesizes two formulas with the same satisfiability and transforms some of the variables. This approach generates a satisfiability-preserving formula that can be used to test SMT solvers. Sparrow [54] leverages logical approximations to mutate a seed formula and obtain satisfiability-preserving SMT instances. However, our approach employs the assignment of a formula to generate additional formulas with known satisfiability. By considering them as basic building blocks along with the sub-formulas of the original formula, we can generate a diverse set of instances from a single seed.

9 Conclusion

In this article, we present a novel method for testing SMT solvers. We propose the utilization of multiple assignments for a formula to evaluate the truth values of its sub-formulas, resulting in the generation of diverse mutants with corresponding test oracles. Furthermore, we leverage these assignments to generate varied formula snippets that participate in the mutation process. Besides, we also incorporate unsatisfiable cores to further adjust the satisfiability of the generated mutants. To validate SMT solvers, we have developed a practical bug detection tool called Octopus. In our experiments, Octopus has successfully identified 11 confirmed bugs in Z3, cvc5, and Bitwuzla, with nine of them being addressed by the respective developers. It is worth noting that among the exposed bugs, multiple bugs were long-latent. Additionally, our evaluation results demonstrate that Octopus significantly outperforms SOTA fuzzers in terms of efficiency, achieved code coverage, and the detection of existing bugs.

Acknowledgments

We thank the anonymous reviewers and editors for their constructive feedback and careful guidance, which substantially improved the quality and clarity of this work. Their comments helped refine both the presentation and the technical content of the article. We are also grateful to the Z3 and cvc5 development teams, particularly Nikolaj Bjørner and Andrew Reynolds, for their generous support and for thoroughly reviewing the bugs we reported. Their insights and engagement were invaluable to this study.

References

[1] Daniel Larraz. 2025. The International SMT Competition. Retrieved from https://github.com/cvc5/cvc5

[2] GNU GCC. 2021. Using the GNU Compiler Collection—GCov. Retrieved from https://gcc.gnu.org/onlinedocs/gcc/Gcov.html

[3] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. 2019. NAUTILUS: Fishing for deep bugs with grammars. In *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS '19)*. The Internet Society. Retrieved from https://www.ndss-symposium.org/ndss-paper/nautilus-fishing-for-deep-bugs-with-grammars/

[4] Haniel Barbosa, Clark W. Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, et al. 2022. cvc5: A versatile and Industrial-Strength SMT solver. In *Proceedings of the 28th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS '22), Held as Part of the European Joint Conferences on Theory and Practice of Software (ETAPS '22)*. Dana Fisman and Grigore Rosu (Eds.), Lecture Notes in Computer Science, Vol. 13243, Springer, 415–442. DOI: https://doi.org/10.1007/978-3-030-99524-9_24

[5] Michael Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, K. Rustan, and M. Leino. 2005. Boogie: A modular reusable verifier for object-oriented programs. In *Proceedings of the 4th International Symposium on Formal Methods for Components and Objects (FMCO '05)*. Frank S. de Boer, Marcello M. Bonsangue, Susanne Graf, and Willem P. de Roever (Eds.), Lecture Notes in Computer Science, Vol. 4111, Springer, 364–387. DOI: https://doi.org/10.1007/11804192_17

[6] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2025. The Satisfiability Modulo Theories Library (SMT-LIB). Retrieved from https://smtlib.cs.uiowa.edu/

[7] Clark Barrett, Aaron Stump, and Cesare Tinelli. 2010. The SMT-LIB standard: Version 2.0. In *Proceedings of the 8th International Workshop on Satisfiability Modulo Theories*.

[8] Clark W. Barrett, Roberto Sebastiani, Sanjit A. Seshia, and Cesare Tinelli. 2009. Satisfiability modulo theories. In *Handbook of Satisfiability*. Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh (Eds.), Frontiers in Artificial Intelligence and Applications, Vol. 185, IOS Press, 825–885. DOI: https://doi.org/10.3233/978-1-58603-929-5-825

[9] Aaron Bembenek, Michael Greenberg, and Stephen Chong. 2020. Formulog: Datalog for SMT-based static analysis. *Proceedings of the ACM on Programming Languages* 4, OOPSLA, Article 141 (2020), 1–31. DOI: https://doi.org/10.1145/3428209

[10] Dmitry Blotsky, Federico Mora, Murphy Berzish, Yunhui Zheng, Ifaz Kabir, and Vijay Ganesh. 2018. StringFuzz: A fuzzer for string solvers. In *Proceedings of the 30th International Conference on Computer Aided Verification (CAV '18)*. Hana Chockler and Georg Weissenbacher (Eds.), Lecture Notes in Computer Science, Vol. 10982, Springer, 45–51. DOI: https://doi.org/10.1007/978-3-319-96142-2_6

[11] Martin Brain. 2023. Random Solutions. Retrieved from https://github.com/cvc5/cvc5/discussions/9581#discussioncomment-5395757

[12] Mauro Bringolf, Dominik Winterer, and Zhendong Su. 2022. Finding and understanding incompleteness bugs in SMT solvers. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. ACM, 1–10. DOI: https://doi.org/10.1145/3551349.3560435

[13] Alexandra Bugariu and Peter Müller. 2020. Automatically testing string solvers. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE '20)*. Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 1459–1470. DOI: https://doi.org/10.1145/3377811.3380398

[14] Cristian Cadar, Daniel Dunbar, and Dawson R. Engler. 2008. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI '08)*. Richard Draves and Robbert van Renesse (Eds.), USENIX Association, 209–224. Retrieved from http://www.usenix.org/events/osdi08/tech/full_papers/cadar/cadar.pdf

[15] Junjie Chen, Wenxiang Hu, Dan Hao, Yingfei Xiong, Hongyu Zhang, Lu Zhang, and Bing Xie. 2016. An empirical comparison of compiler testing techniques. In *Proceedings of the 38th International Conference on Software Engineering (ICSE '16)*. Laura K. Dillon, Willem Visser, and Laurie A. Williams (Eds.), ACM, 180–190. DOI: https://doi.org/10.1145/2884781.2884878

[16] Tsong Yueh Chen, S. C. Cheung, and Siu-Ming Yiu. 2020. Metamorphic testing: A new approach for generating next test cases. arXiv:2002.12543. Retrieved from https://arxiv.org/abs/2002.12543

[17] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. 2018. Metamorphic testing: A review of challenges and opportunities. *ACM Comput. Surv.* 51, 1, Article 4 (2018), 1–27. DOI: https://doi.org/10.1145/3143561

[18] Leonardo Mendonça de Moura and Nikolaj S. Bjørner. 2008. Z3: An efficient SMT solver. In *Proceedings of the 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS '08), Held as Part of the Joint European Conferences on Theory and Practice of Software (ETAPS '08)*. C. R. Ramakrishnan and Jakob Rehof (Eds.), Lecture Notes in Computer Science, Vol. 4963, Springer, 337–340. DOI: https://doi.org/10.1007/978-3-540-78800-3_24

[19] Patrice Godefroid, Michael Y. Levin, and David A. Molnar. 2008. Automated whitebox fuzz testing. In *Proceedings of the Network and Distributed System Security Symposium (NDSS '08)*. The Internet Society. Retrieved from https://www.ndss-symposium.org/ndss2008/automated-whitebox-fuzz-testing/

[20] HyungSeok Han, DongHyeon Oh, and Sang Kil Cha. 2019. CodeAlchemist: Semantics-Aware code generation to find vulnerabilities in JavaScript engines. In *Proceedings of the Network and Distributed System Security Symposium (NDSS '19)*. The Internet Society.

[21] Heqing Huang. 2023. CVE-2020-19725. Retrieved November 5, 2023 from https://nvd.nist.gov/vuln/detail/CVE-2020-19725#

[22] Mingyue Jiang, Tsong Yueh Chen, Fei-Ching Kuo, and Zuohua Ding. 2013. Testing central processing unit scheduling algorithms using metamorphic testing. In *Proceedings of the 2013 IEEE 4th International Conference on Software Engineering and Service Science*, 530–536. DOI: https://doi.org/10.1109/ICSESS.2013.6615365

[23] Jongwook Kim, Sunbeom So, and Hakjoo Oh. 2023. Diver: Oracle-guided SMT solver testing with unrestricted random mutations. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 2224–2236. DOI: https://doi.org/10.1109/ICSE48619.2023.00187

[24] Gereon Kremer, Aina Niemetz, and Mathias Preiner. 2021. ddSMT 2.0: Better delta debugging for the SMT-LIBv2 language and friends. In *Proceedings of the 33rd International Conference on Computer Aided Verification (CAV '21)*. Alexandra Silva and K. R. M. Leino (Eds.), Lecture Notes in Computer Science, Vol. 12760, Springer, 231–242. DOI: https://doi.org/10.1007/978-3-030-81688-9_11

[25] Fei-Ching Kuo, Tsong Yueh Chen, and Wing K. Tam. 2011. Testing embedded software by metamorphic testing: A wireless metering system case study. In *Proceedings of the IEEE 36th Conference on Local Computer Networks (LCN '11)*. Chun Tung Chou, Tom Pfeifer, and Anura P. Jayasumana (Eds.), IEEE Computer Society, 291–294. DOI: https://doi.org/10.1109/LCN.2011.6115306

[26] K. Rustan M. Leino. 2010. Dafny: An automatic program verifier for functional correctness. In *Proceedings of the 16th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning (LPAR '10)*. Edmund M. Clarke and Andrei Voronkov (Eds.), Lecture Notes in Computer Science, Vol. 6355, Springer, 348–370. DOI: https://doi.org/10.1007/978-3-642-17511-4_20

[27] H. B. Mann and D. R. Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60. DOI: https://doi.org/10.1214/aoms/1177730491

[28] Muhammad Numair Mansur, Maria Christakis, Valentin Wüstholz, and Fuyuan Zhang. 2020. Detecting critical bugs in SMT solvers using blackbox mutational fuzzing. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.), ACM, 701–712. DOI: https://doi.org/10.1145/3368089.3409763

[29] Christian Murphy, Gail E. Kaiser, Lifeng Hu, and Leon Wu. 2008. Properties of machine learning applications for use in metamorphic testing. In *Proceedings of the 20th International Conference on Software Engineering and Knowledge Engineering (SEKE ' 08)*. Knowledge Systems Institute Graduate School, 867–872.

[30] Aina Niemetz, Mathias Preiner, and Clark W. Barrett. 2022. Murxla: A modular and highly extensible API fuzzer for SMT solvers. In *Proceedings of the 34th International Conference on Computer Aided Verification (CAV '22)*. Sharon Shoham and Yakir Vizel (Eds.), Lecture Notes in Computer Science, Vol. 13372, Springer, 92–106. DOI: https://doi.org/10.1007/978-3-031-13188-2_5

[31] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. 2018. Btor2, BtorMC and boolector 3.0. In *Proceedings of the 30th International Conference on Computer Aided Verification (CAV '18), Held as Part of the Federated Logic Conference (FloC '18)*, 587–595. DOI: https://doi.org/10.1007/978-3-319-96145-3_32

[32] Jiwon Park, Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2021. Generative type-aware mutation for testing SMT solvers. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021), 1–19. DOI: https://doi.org/10.1145/3485529

[33] Van-Thuan Pham, Marcel Boehme, Andrew Edward Santosa, Alexandru Razvan Caciulescu, and Abhik Roychoudhury. 2021. Smart greybox fuzzing. *IEEE Trans. Software Eng.* 47, 9 (2021), 1980–1997. DOI: https://doi.org/10.1109/TSE.2019.2941681

[34] Joseph Scott, Trishal Sudula, Hammad Rehman, Federico Mora, and Vijay Ganesh. 2021. BanditFuzz: Fuzzing SMT solvers with multi-agent reinforcement learning. In *Proceedings of the 24th International Symposium on Formal Methods (FM '21)*. Marieke Huisman, Corina S. Pasareanu, and Naijun Zhan (Eds.), Lecture Notes in Computer Science, Vol. 13047. Springer, 103–121. DOI: https://doi.org/10.1007/978-3-030-90870-6_6

[35] Sergio Segura, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortes. 2016. A survey on metamorphic testing. *IEEE Transactions on Software Engineering* 42, 9 (2016), 805–824. DOI: https://doi.org/10.1109/TSE.2016.2532875

[36] Daniel Selsam and Nikolaj S. Bjørner. 2019. Guiding high-performance SAT solvers with unsat-core predictions. In *Proceedings of the 22nd International Conference on Theory and Applications of Satisfiability Testing (SAT '19)*. Mikolás Janota and Inês Lynce (Eds.), Lecture Notes in Computer Science, Vol. 11628, Springer, 336–353. DOI: https://doi.org/10.1007/978-3-030-24258-9_24

[37] SMT-COMP. 2021. The International SMT Competition. Retrieved from https://smt-comp.github.io/2021/index.html

[38] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding compiler bugs via live code mutation. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '16)*. Eelco Visser and Yannis Smaragdakis (Eds.), ACM, 849–863. DOI: https://doi.org/10.1145/2983990.2984038

[39] Maolin Sun, Yibiao Yang, Yang Wang, Ming Wen, Haoxiang Jia, and Yuming Zhou. 2023. SMT solver validation empowered by large pre-trained language models. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 1288–1300. DOI: https://doi.org/10.1109/ASE56229.2023.00180

[40] Maolin Sun, Yibiao Yang, Ming Wen, Yongcong Wang, Yuming Zhou, and Hai Jin. 2023. Validating SMT solvers via skeleton enumeration empowered by historical bug-triggering inputs. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 69–81. DOI: https://doi.org/10.1109/ICSE48619.2023.00018

[41] Maolin Sun, Yibiao Yang, Jiangchang Wu, and Yuming Zhou. 2025. Validating SMT rewriters via rewrite space exploration supported by generative equality saturation. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2, Article 315 (Oct. 2025), 1–27. DOI: https://doi.org/10.1145/3763093

[42] Maolin Sun, Yibiao Yang, and Yuming Zhou. 2025. Boosting skeleton-driven SMT solver fuzzing by leveraging LLM to produce formula generators. arXiv:2508.20340. Retrieved from https://arxiv.org/abs/2508.20340

[43] Yixuan Tang, He Jiang, Zhide Zhou, Xiaochen Li, Zhilei Ren, and Weiqiang Kong. 2022. Detecting compiler warning defects via diversity-guided program mutation. *IEEE Transactions on Software Engineering* 48, 11 (2022), 4411–4432. DOI: https://doi.org/10.1109/TSE.2021.3119186

[44] Dave Towey, Yunwei Dong, Chang-Ai Sun, and Tsong Yueh Chen. 2016. Metamorphic testing as a test case selection strategy. *Science China Information Sciences* 59, 5 (2016), 050108. DOI: https://doi.org/10.1007/s11432-016-5544-6

[45] David Trabish, Shachar Itzhaky, and Noam Rinetzky. 2021. A bounded symbolic-size model for symbolic execution. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. Diomidis Spinellis, Georgios Gousios, Marsha Chechik, and Massimiliano Di Penta (Eds.), ACM, 1190–1201. DOI: https://doi.org/10.1145/3468264.3468596

[46] Dominik Winterer and Zhendong Su. 2024. Validating SMT solvers for correctness and performance via grammar-based enumeration. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 2378–2401. DOI: https://doi.org/10.1145/3689795

[47] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. On the unusual effectiveness of type-aware operator mutations for testing SMT solvers. *Proceedings of the ACM on Programming Languages* 4, OOPSLA, Article 193 (2020), 1–25. DOI: https://doi.org/10.1145/3428261

[48] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. Validating SMT solvers via semantic fusion. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '20)*. Alastair F. Donaldson and Emina Torlak (Eds.), ACM, 718–730. DOI: https://doi.org/10.1145/3385412.3385985

[49] Xiaoyuan Xie, Joshua Wing Kei Ho, Christian Murphy, Gail E. Kaiser, Baowen Xu, and Tsong Yueh Chen. 2009. Application of metamorphic testing to supervised classifiers. In *Proceedings of the 9th International Conference on Quality Software (QSIC '09)*. Byoungju Choi (Ed.), IEEE Computer Society, 135–144. DOI: https://doi.org/10.1109/QSIC.2009.26

[50] Yibiao Yang, Yanyan Jiang, Zhiqiang Zuo, Yang Wang, Hao Sun, Hongmin Lu, Yuming Zhou, and Baowen Xu. 2019. Automatic self-validation for code coverage profilers. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE '19)*. IEEE, 79–90. DOI: https://doi.org/10.1109/ASE.2019.00018

[51] Yibiao Yang, Maolin Sun, Yang Wang, Qingyang Li, Ming Wen, and Yuming Zhou. 2023. Heterogeneous testing for coverage profilers empowered with debugging support. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.), ACM, 670–681. DOI: https://doi.org/10.1145/3611643.3616340

[52] Yibiao Yang, Maolin Sun, Jiangchang Wu, Qingyang Li, and Yuming Zhou. 2025. Debugger toolchain validation via cross-level debugging. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*. Lieven Eeckhout, Georgios Smaragdakis, Kaitai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.), ACM, 280–294. DOI: https://doi.org/10.1145/3669940.3707271

[53] Peisen Yao, Heqing Huang, Wensheng Tang, Qingkai Shi, Rongxin Wu, and Charles Zhang. 2021. Fuzzing SMT solvers via two-dimensional input space exploration. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '21)*. Cristian Cadar and Xiangyu Zhang (Eds.), ACM, 322–335. DOI: https://doi.org/10.1145/3460319.3464803

[54] Peisen Yao, Heqing Huang, Wensheng Tang, Qingkai Shi, Rongxin Wu, and Charles Zhang. 2021. Skeletal approximation enumeration for SMT solver testing. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. Diomidis Spinellis, Georgios Gousios, Marsha Chechik, and Massimiliano Di Penta (Eds.), ACM, 1141–1153. DOI: https://doi.org/10.1145/3468264.3468540

[55] Michal Zalewski. 2014. American Fuzzy Lop. Retrieved from http://lcamtuf.coredump.cx/afl/

[56] Yufeng Zhang, Zhenbang Chen, Ziqi Shuai, Tianqi Zhang, Kenli Li, and Ji Wang. 2020. Multiplex symbolic execution: Exploring multiple paths by solving once. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*. IEEE, 846–857. DOI: https://doi.org/10.1145/3324884.3416645

[57] Yao Zhang, Xiaofei Xie, Yi Li, Yun Lin, Sen Chen, Yang Liu, and Xiaohong Li. 2023. Demystifying performance regressions in string solvers. *IEEE Transactions on Software Engineering* 49, 3 (2023), 947–961. DOI: https://doi.org/10.1109/TSE.2022.3168373

[58] Zhiquan Zhou, Shujia Zhang, Markus Hagenbuchner, T. H. Tse, Fei-Ching Kuo, and Tsong Yueh Chen. 2012. Automated functional testing of online search services. *Software Testing, Verification and Reliability* 22, 4 (2012), 221–243. DOI: https://doi.org/10.1002/stvr.437

Received 25 November 2024; revised 2 September 2025; accepted 7 January 2026